

Chapter 13. Putting It All Together: Experimenting and Ranking

In the last few chapters, we have covered many aspects of ranking, including various kinds of loss functions as well as metrics for measuring the performance of ranking systems. In this chapter, we will show an example of a ranking loss and ranking metric on the [Spotify Million Playlist dataset](#).

This chapter encourages a lot more experimentation and is more open-ended than the previous ones, whose goal was to introduce concepts and infrastructure. This chapter, on the other hand, is written to encourage you to roll up your sleeves and engage directly with loss functions and writing metrics.

Experimentation Tips

Before we begin digging into the data and modeling, let's cover some practices that will make your life easier when doing a lot of experimentation and rapid iteration. These are general guidelines that have made our experimentation faster. As a result, we're able to rapidly iterate toward solutions that help us reach our objectives.

Experimental code is different from engineering code in that the code is written to explore ideas, not for robustness. The goal is to achieve maximum velocity while not sacrificing too much in terms of code quality. So you should think about whether a piece of code should be thoroughly tested or whether this isn't necessary because the code is present only to test a hypothesis and then it will be thrown away. With that in mind, here are some tips. Keep in mind that these tips are the opinion of the authors,

developed over time, and are not hard-and-fast rules, just some flavored opinions that some may disagree with.

Keep It Simple

In terms of the overall structure of research code, it's best to keep it as simple as possible. Try not to overthink too much in terms of inheritance and reusability during the early stages of the lifecycle of exploration. At the start of a project, we usually don't know what it needs yet, so the preference should be keeping the code easily readable and simple for debugging. That means you don't have to focus too much on code reuse because at the early stage of a project, many code changes will occur while the structure of the model, data ingestion, and interaction of various parts of a system are being worked out. When the uncertainties have been worked out, then you can rewrite the code into a more robust form, but refactoring too early actually slows velocity.

A general rule of thumb is that it is OK to copy code three times and then refactor out into a library the fourth time, because you'll have seen enough use cases to justify the reuse of code. If refactoring is done too early, you might not have seen enough use cases of a piece of code to cover the possible use cases that it might need to handle.

Debug Print Statements

If you've read a number of ML research papers, you may expect your data to be fairly clean and orderly at the start of a project. However, real-world data can be messy, with missing fields and unexpected values. Having lots of print functions allows you to print and visually inspect a sample of the data and also helps in crafting the input data pipelines and transformations to feed the model. Also, printing sample outputs of the model is useful in making sure the output is as expected.

The most important places to include logging are the input and output schema between components of your system; these help you understand where reality may be deviating from expectations. Later, you can make unit tests to ensure that refactoring of the model doesn't break anything, but the unit tests can wait for when the model architecture is stable. A

good rule of thumb is to add unit tests when you want to refactor code or reuse or optimize the code to preserve functionality or when the code is stable and you want to ensure that it doesn't break a build. Another good use case of adding print statements is when you inevitably run into not-a-number (NaN) errors when running training code.

In JAX, you can enable NaN debugging by using the following lines:

```
from jax import config
config.update("jax_debug_nans", True)

@jax.jit
def f(x):
    jax.debug.print("Debugging {x}", x=x)
```

The debug NaNs configuration setting will rerun a jitted function if it finds any NaNs, and the debug print function will print the value of the tensors even inside a JIT. A regular print won't work inside a JIT because it is not a compilable command and is skipped over during the tracing, so you have to use the debug print function instead, which does work inside a JIT.

Defer Optimization

In research code, there is a lot of temptation to optimize early—in particular, focusing on the implementation of your models or system to ensure they're efficient computationally or the code is elegant. However, research code is written for higher velocity in experimentation, not execution speed.

Our suggestion is do not optimize too early unless it hinders research velocity. One reason for this is the system might not be complete, so optimizing one part might not make sense if another part of the system is even slower and is the actual bottleneck. Another reason is the part that you are optimizing might not make it to the final model, so all the optimization work might go to waste if the code is refactored away anyway.

Finally, optimization might actually hinder the ability to modify or inject newer design choices in terms of architecture or functionality. Optimized code tends to have certain choices that were made that fit the current structure of the data flow but might not be amenable to further changes. For example, in the code for this chapter, one possible optimization choice would have been to batch together playlists of the same size so that the code might be able to run in larger batches. However, at this point of the experimentation, that optimization would have been premature and distracting because it might make the metrics code more complicated. Our gentle advice is to defer optimization until after the bulk of experimentation has been done and the architecture, loss functions, and metrics have been chosen and settled upon.

Keep Track of Changes

In research code, too many variables are probably at play for you to change them one at a time to see their effects. This problem is particularly noticeable with larger datasets that require a lot of runs to determine which change causes which effects. So, in general, fixing a number of parameters and changing the code bit by bit is still a good idea so that you can keep track of the change that causes the most improvement. Parameters have to be tracked, but so do the code changes.

One way to keep track of changes is through services such as Weights & Biases that we discussed in [Chapter 5](#). Keeping track of the exact code that led to a change and the parameters is a good idea so that experiments can be reproduced and analyzed. Especially with research code that changes so frequently and is sometimes not checked in, you have to be diligent in keeping a copy of the code that produced a run somewhere, and MLOps tools allow you to track code and hyperparameters.

Use Feature Engineering

Unlike in academic papers, most applied research is interested in a good outcome rather than a theoretically beautiful result. We're not shackled by purist views that the model has to learn everything about the data by itself. Instead, we're pragmatic and concerned about good outcomes.

We should not discard practices like feature engineering, especially when we have little data or are crunched for time and need decent results fast. Using feature engineering means that if you know whether a handcrafted feature is correlated positively or negatively with an outcome like the ranking of an item, then by all means add these engineered features to the data. An example in recommender systems is having an attribute of the item being scored that matches something in the user’s profile. So, if an item has the same artist or album in the user’s playlist, we can return a Boolean True; otherwise, we return False. This extra feature simply helps the model converge faster, and the model can still use other latent features such as embeddings to compensate if the hand-engineered features don’t do so well.

It is generally a good practice to ablate the hand-engineered features once in a while. To do this, hold back an experiment without some features to see if those features have become obsolete over time or if they still benefit the business metrics.

ABLATION

Ablation in ML applications is the practice of measuring the change in performance of a model when a particular feature is removed. In computer vision applications, ablation often refers to blocking part of the image or view field to see how it impacts the model’s ability to identify or segment data. In other kinds of ML, it can mean strategically removing certain features.

One gotcha with ablation is what to replace the feature with. Simply *zeroing out* the feature can significantly skew the output of the model. This is called *zero-ablation*, and can force the model to treat that feature out of distribution, which yields less believable outcomes. Instead, some advocate for mean-ablation, or taking the average or most common value of that feature. This allows the model to see much more expected values, and reduce these risks.

However, this fails to consider the most important aspects of the kinds of models we’ve been working on—latent high-order interactions. One of the authors has investigated a deeper approach to ablation called *causal scrubbing*, in which you fix the ablation value to be sampled from the posterior distribution produced by other feature values, i.e., a value that “makes sense” with the rest of the values the model will see at that time.

Understand Metrics Versus Business Metrics

Sometimes, as ML practitioners, we obsess over the best possible metrics our models can achieve. However, we should temper that enthusiasm as the best ML metric might not totally represent the business interests at hand. Furthermore, other systems that contain business logic might sit on top of our models and modify the output. As a result, it is best not to obsess too heavily over ML metrics and to do proper A/B tests that contain business metrics instead since that's the main measure of a good outcome with ML.

The best possible circumstance is to find a loss function that aligns well or predicts the relevant business metric. This, unfortunately, is often not easy to find, especially when the business metrics are nuanced or have competing priorities.

Perform Rapid Iteration

Don't be afraid to look at results of runs that are rather short. There's no need to do a full pass over the data at the beginning, when you are figuring out the interaction between a model architecture and the data. It's OK to do some rapid runs with minor tweaks to see how they change the metrics over a short number of time steps. In the Spotify Million Playlist dataset, we tweaked the model architecture by using 100,000 playlists before doing longer runs. Sometimes the changes can be so dramatic that the effects can be seen immediately, even at the first test-set evaluation.

Now that we have the basics of experimental research coding covered, let's hop over to the data and code and play a bit with modeling music recommendations.

Spotify Million Playlist Dataset

The code for this section can be found in [this book's GitHub repo](#). The documentation for the data can be found at [Spotify Million Playlist Dataset Challenge](#).

The first thing we should do is take a look at the data:

```
less data/spotify_million_playlist_dataset/data/mpd.slice.0-999.json
```

That should produce the following output:

```
{
  "info": {
    "generated_on": "2017-12-03 08:41:42.057563",
    "slice": "0-999",
    "version": "v1"
  },
  "playlists": [
    {
      "name": "Throwbacks",
      "collaborative": "false",
      "pid": 0,
      "modified_at": 1493424000,
      "num_tracks": 52,
      "num_albums": 47,
      "num_followers": 1,
      "tracks": [
        {
          "pos": 0,
          "artist_name": "Missy Elliott",
          "track_uri": "spotify:track:0UaMYEvWZi0ZqiD0oHU3YI",
          "artist_uri": "spotify:artist:2wIVse2owClT7go1WT98tk",
          "track_name": "Lose Control (feat. Ciara & Fat Man Scoop)",
          "album_uri": "spotify:album:6vV5UrXcfyQD1wu4Qo2I9K",
          "duration_ms": 226863,
          "album_name": "The Cookbook"
        }
      ]
    }
  ]
}
```

When encountering a new dataset, it is always important to look at it and plan which features to use to generate recommendations for the data.

One possible goal of the Spotify Million Playlist Dataset Challenge is to see if the next tracks in a playlist can be predicted from the first five tracks in the playlist.

In this case, several features might be useful for the task. We have track, artist, and album universal resource identifiers (URIs), which are unique identifiers for tracks, artists, and albums, respectively. And we have artist and album names and names of playlists. The dataset also includes numerical features like duration of a track and the number of followers in a playlist. Intuitively, the number of followers of a playlist should not affect the ordering of tracks in a playlist, so you might want to look for better features before using these possibly uninformative ones. Looking at the overall statistics of features, you can also obtain a lot of insight:

```
less data/spotify_million_playlist_dataset/stats.txt
number of playlists 1000000
number of tracks 66346428
number of unique tracks 2262292
number of unique albums 734684
number of unique artists 295860
number of unique titles 92944
number of playlists with descriptions 18760
number of unique normalized titles 17381
avg playlist length 66.346428

top playlist titles
10000 country
10000 chill
8493 rap
8481 workout
8146 oldies
8015 christmas
6848 rock
6157 party
5883 throwback
5063 jams
5052 worship
4907 summer
4677 feels
4612 new
4186 disney
4124 lit
4030 throwbacks
```

First of all, notice that the number of tracks is more than the number of playlists. This implies that quite a few tracks might have very little training data. So the `track_uri` might not be a feature that generalizes very well. On the other hand, the `album_uri` and `artist_uri` would generalize because they would occur multiple times in different playlists. For the sake of code clarity, we will mostly work with the `album_uri` and `artist_uri` as the features that represent a track.

In previous “Putting It All Together” chapters, we demonstrated the use of content-based features or text token-based features that may be used instead, but direct embedding features are the clearest for demonstrating ranking. In a real-world application, embedding features and content-based features may be concatenated together to form a feature that generalizes better for recommendation ranking. For the purposes of this chapter, we will represent a track as the tuple of (`track_id`, `album_id`, `artist_id`), where the ID is an integer representing the URI. We will build dictionaries that map from the URI to the integer ID in the next section.

Building URI Dictionaries

Similar to [Chapter 8](#), we will first start by constructing a dictionary for all the URIs. This dictionary allows us to represent the text URI as an integer for faster processing on the JAX side, as we can easily look up embeddings from integers as opposed to arbitrary URI strings.

Here is the code for *make_dictionary.py*:

```
import glob
import json
import os
from typing import Any, Dict, Tuple

from absl import app
from absl import flags
from absl import logging
import numpy as np
import tensorflow as tf

FLAGS = flags.FLAGS
```

```

_PLAYLISTS = flags.DEFINE_string("playlists", None, "Playlist json glob.")
_OUTPUT_PATH = flags.DEFINE_string("output", "data", "Output path.")

# Required flag.
flags.mark_flag_as_required("playlists")

def update_dict(dict: Dict[Any, int], item: Any):
    """Adds an item to a dictionary."""
    if item not in dict:
        index = len(dict)
        dict[item] = index

def dump_dict(dict: Dict[str, str], name: str):
    """Dumps a dictionary as json."""
    fname = os.path.join(_OUTPUT_PATH.value, name)
    with open(fname, "w") as f:
        json.dump(dict, f)

def main(argv):
    """Main function."""
    del argv # Unused.

    tf.config.set_visible_devices([], 'GPU')
    tf.compat.v1.enable_eager_execution()
    playlist_files = glob.glob(_PLAYLISTS.value)
    track_uri_dict = {}
    artist_uri_dict = {}
    album_uri_dict = {}

    for playlist_file in playlist_files:
        print("Processing ", playlist_file)
        with open(playlist_file, "r") as file:
            data = json.load(file)
            playlists = data["playlists"]
            for playlist in playlists:
                tracks = playlist["tracks"]
                for track in tracks:
                    update_dict(track_uri_dict, track["track_uri"])
                    update_dict(artist_uri_dict, track["artist_uri"])
                    update_dict(album_uri_dict, track["album_uri"])

    dump_dict(track_uri_dict, "track_uri_dict.json")
    dump_dict(artist_uri_dict, "artist_uri_dict.json")
    dump_dict(album_uri_dict, "album_uri_dict.json")

```

```
if __name__ == "__main__":  
    app.run(main)
```

Whenever a new URI is encountered, we simply increment a counter and assign that unique identifier to the URI. We do this for tracks, artists, and albums and save it as a JSON file.

Although we could have used a data processing framework like PySpark for this, it is important to take note of the data size. If the data size is small, like a million playlists, it would just be faster to do it on a single machine. We should be wise about when to use a big data processing framework, and for small datasets it can sometimes be faster to simply run the code on one machine instead of writing code that runs on a cluster.

Building the Training Data

Now that we have the dictionaries, we can use them to convert the raw JSON playlist logs into a more usable form for ML training. The code for this is in *make_training.py*:

```
import glob  
import json  
import os  
from typing import Any, Dict, Tuple  
  
from absl import app  
from absl import flags  
from absl import logging  
import numpy as np  
import tensorflow as tf  
  
import input_pipeline  
  
FLAGS = flags.FLAGS  
_PLAYLISTS = flags.DEFINE_string("playlists", None, "Playlist json glob.")  
_DICTIONARY_PATH = flags.DEFINE_string("dictionaries", "data/dictionaries",  
                                       "Dictionary path.")  
_OUTPUT_PATH = flags.DEFINE_string("output", "data/training", "Output path.")
```

```

_TOP_K = flags.DEFINE_integer("topk", 5, "Top K tracks to use as context.")
_MIN_NEXT = flags.DEFINE_integer("min_next", 10, "Min number of tracks.")

# Required flag.
flags.mark_flag_as_required("playlists")

def main(argv):
    """Main function."""
    del argv # Unused.

    tf.config.set_visible_devices([], 'GPU')
    tf.compat.v1.enable_eager_execution()
    playlist_files = glob.glob(_PLAYLISTS.value)

    track_uri_dict = input_pipeline.load_dict(
        _DICTIONARY_PATH.value, "track_uri_dict.json")

    print("%d tracks loaded" % len(track_uri_dict))
    artist_uri_dict = input_pipeline.load_dict(
        _DICTIONARY_PATH.value, "artist_uri_dict.json")
    print("%d artists loaded" % len(artist_uri_dict))
    album_uri_dict = input_pipeline.load_dict(
        _DICTIONARY_PATH.value, "album_uri_dict.json")
    print("%d albums loaded" % len(album_uri_dict))
    topk = _TOP_K.value
    min_next = _MIN_NEXT.value
    print("Filtering out playlists with less than %d tracks" % min_next)

    raw_tracks = {}

    for pidx, playlist_file in enumerate(playlist_files):
        print("Processing ", playlist_file)
        with open(playlist_file, "r") as file:
            data = json.load(file)
            playlists = data["playlists"]
            tfrecord_name = os.path.join(
                _OUTPUT_PATH.value, "%05d.tfrecord" % pidx)
            with tf.io.TFRecordWriter(tfrecord_name) as file_writer:
                for playlist in playlists:
                    if playlist["num_tracks"] < min_next:
                        continue
                    tracks = playlist["tracks"]
                    # The first topk tracks are all for the context.
                    track_context = []

```

```

artist_context = []
album_context = []
# The rest are for predicting.
next_track = []
next_artist = []
next_album = []
for tidx, track in enumerate(tracks):
    track_uri_idx = track_uri_dict[track["track_uri"]]
    artist_uri_idx = artist_uri_dict[track["artist_uri"]]
    album_uri_idx = album_uri_dict[track["album_uri"]]
    if track_uri_idx not in raw_tracks:
        raw_tracks[track_uri_idx] = track
    if tidx < topk:
        track_context.append(track_uri_idx)
        artist_context.append(artist_uri_idx)
        album_context.append(album_uri_idx)
    else:
        next_track.append(track_uri_idx)
        next_artist.append(artist_uri_idx)
        next_album.append(album_uri_idx)
assert(len(next_track) > 0)
assert(len(next_artist) > 0)
assert(len(next_album) > 0)
record = tf.train.Example(
    features=tf.train.Features(feature={
        "track_context": tf.train.Feature(
            int64_list=tf.train.Int64List(value=track_context)),
        "album_context": tf.train.Feature(
            int64_list=tf.train.Int64List(value=album_context)),
        "artist_context": tf.train.Feature(
            int64_list=tf.train.Int64List(value=artist_context)),
        "next_track": tf.train.Feature(
            int64_list=tf.train.Int64List(value=next_track)),
        "next_album": tf.train.Feature(
            int64_list=tf.train.Int64List(value=next_album)),
        "next_artist": tf.train.Feature(
            int64_list=tf.train.Int64List(value=next_artist)),
    }))
record_bytes = record.SerializeToString()
file_writer.write(record_bytes)

```

```

filename = os.path.join(_OUTPUT_PATH.value, "all_tracks.json")
with open(filename, "w") as f:
    json.dump(raw_tracks, f)

```

```
if __name__ == "__main__":  
    app.run(main)
```

This code reads in a raw playlist JSON file, converts the URIs from textual identifiers to the index in the dictionary, and filters out playlists that are under a minimum size. In addition, we partition the playlist such that the first five elements are grouped into the context, or user that we are recommending items for, and the next items, which are the items we wish to predict for a given user. We call the first five elements the *context* because they represent a playlist and because there won't be a one-to-one mapping between a playlist and a user if a user has more than one playlist. We then write each playlist as a TensorFlow example in a TensorFlow record file for use with the TensorFlow data input pipeline. The records will always contain five tracks, albums, and artists for the context and at least five more next tracks for learning the inference tasks of predicting the next tracks.

NOTE

We use TensorFlow objects here because of their compatibility with JAX and to introduce some very convenient data formats.

We also store unique rows of tracks with all the features, which is mostly for debugging and display should we need to convert a `track_uri` into a human-readable form. This track data is stored in *all_tracks.json*.

Reading the Input

The input is then read via *input_pipeline.py*:

```
import glob  
import json  
import os  
from typing import Sequence, Tuple, Set  
  
import tensorflow as tf  
import jax.numpy as jnp
```

```

_schema = {
    "track_context": tf.io.FixedLenFeature([5], dtype=tf.int64),
    "album_context": tf.io.FixedLenFeature([5], dtype=tf.int64),
    "artist_context": tf.io.FixedLenFeature([5], dtype=tf.int64),
    "next_track": tf.io.VarLenFeature(dtype=tf.int64),
    "next_album": tf.io.VarLenFeature(dtype=tf.int64),
    "next_artist": tf.io.VarLenFeature(dtype=tf.int64),
}

def _decode_fn(record_bytes):
    result = tf.io.parse_single_example(record_bytes, _schema)
    for key in _schema.keys():
        if key.startswith("next"):
            result[key] = tf.sparse.to_dense(result[key])
    return result

def create_dataset(
    pattern: str):
    """Creates a spotify dataset.

    Args:
        pattern: glob pattern of tfrecords.
    """
    filenames = glob.glob(pattern)
    ds = tf.data.TFRecordDataset(filenames)
    ds = ds.map(_decode_fn)
    return ds

```

We use the TensorFlow data's functionality to read and decode the TensorFlow records and examples. For that to work, we need to supply a schema, or a dictionary, telling the decoder the names and types of features to expect. Since we have picked five tracks each for the context, we should expect five each of `track_context`, `album_context`, and `artist_context`. However, since the playlists themselves are of variable lengths, we tell the decoder to expect variable-length integers for the `next_track`, `next_album`, and `next_artist` features.

The second part of *input_pipeline.py* is for reusable input code to load the dictionaries and track metadata:

```

def load_dict(dictionary_path: str, name: str):
    """Loads a dictionary."""
    filename = os.path.join(dictionary_path, name)
    with open(filename, "r") as f:
        return json.load(f)

def load_all_tracks(all_tracks_file: str,
                    track_uri_dict, album_uri_dict, artist_uri_dict):
    """Loads all tracks.

    """
    with open(all_tracks_file, "r") as f:
        all_tracks_json = json.load(f)
    all_tracks_dict = {
        int(k): v for k, v in all_tracks_json.items()
    }
    all_tracks_features = {
        k: (track_uri_dict[v["track_uri"]],
            album_uri_dict[v["album_uri"]],
            artist_uri_dict[v["artist_uri"]])
        for k, v in all_tracks_dict.items()
    }
    return all_tracks_dict, all_tracks_features

def make_all_tracks_numpy(all_tracks_features):
    """Makes the entire corpus available for scoring."""
    all_tracks = []
    all_albums = []
    all_artists = []
    items = sorted(all_tracks_features.items())
    for row in items:
        k, v = row
        all_tracks.append(v[0])
        all_albums.append(v[1])
        all_artists.append(v[2])
    all_tracks = jnp.array(all_tracks, dtype=jnp.int32)
    all_albums = jnp.array(all_albums, dtype=jnp.int32)
    all_artists = jnp.array(all_artists, dtype=jnp.int32)
    return all_tracks, all_albums, all_artists

```

We also supply a utility function to convert the *all_tracks.json* file into the entire corpus of tracks for scoring in the final recommendations. After all,

the goal is to rank the entire corpus, given the first five context tracks, and see how well they match the given next track data.

Modeling the Problem

Next, let's think of how we will model the problem. We have five context tracks, each with an associated artist and album. We know that we have more tracks than playlists, so for now we will simply ignore the `track_id` and just use the `album_id` and `artist_id` as features. One strategy could be to use one-hot encoding for the album and artist, and this would work well, but one-hot encoding tends to lead to models with high precision but less generalization.

An alternate way to represent identifiers is to embed them—that is, to make a lookup table to an embedding of a fixed size that is lower dimensional than the cardinality of the identifiers. This embedding can be thought of as a low-rank approximation to the full-rank matrix of identifiers. We covered low-rank embeddings in earlier chapters, and we use that concept here as features to represent the album and artists.

Take a look at *models.py*, which contains the code for `SpotifyModel`:

```
from functools import partial
from typing import Any, Callable, Sequence, Tuple

from flax import linen as nn
import jax.numpy as jnp

class SpotifyModel(nn.Module):
    """Spotify model that takes a context and predicts the next tracks."""
    feature_size : int

    def setup(self):
        # There are too many tracks and albums so limit by hashing.
        self.max_albums = 100000
        self.album_embed = nn.Embed(self.max_albums, self.feature_size)
        self.artist_embed = nn.Embed(295861, self.feature_size)

    def get_embeddings(self, album, artist):
        """
```

```

    Given track, album, artist indices return the embeddings.
    Args:
        album: ints of shape nx1
        artist: ints of shape nx1
    Returns:
        Embeddings representing the track.
    """
    album_modded = jnp.mod(album, self.max_albums)
    album_embed = self.album_embed(album_modded)
    artist_embed = self.artist_embed(artist)
    result = jnp.concatenate([album_embed, artist_embed], axis=-1)
    return result

```

In the setup code, notice that we have two embeddings, for the albums and the artists. We have a lot of albums, so we show one way to reduce the memory footprint of album embeddings: take the mod of a smaller number than the number of embeddings so that multiple albums might share an embedding. If more memory is available, you can remove the mod, but this technique is demonstrated here as a way of getting some benefit of having an embedding for a feature with very large cardinality.

The artist is probably the most informative feature, and the data includes far fewer unique artists, so we have a one-to-one mapping between the `artist_id` and the embeddings. When we convert the tuple of `(album_id, artist_id)` to an embedding, we do separate lookups for each ID and then concatenate the embeddings and return one complete embedding to represent a track. If more playlist data becomes available, you might also want to embed the `track_id`. However, given that we have more unique tracks than playlists, the `track_id` feature will not generalize well until we have more playlist data and the `track_id` could occur more often as observations. A general rule of thumb is that a feature should occur at least 100 times to be useful; otherwise, the gradients for that feature will not be updated very often, and it might as well be a random number because it is initialized as such.

In the `call` section, we do the heavy lifting of computing the affinity of a context to other tracks:

```

def __call__(self,
              track_context, album_context, artist_context,
              next_track, next_album, next_artist,
              neg_track, neg_album, neg_artist):
    """Returns the affinity score to the context.
    Args:
        track_context: ints of shape n
        album_context: ints of shape n
        artist_context: ints of shape n
        next_track: int of shape m
        next_album: int of shape m
        next_artist: int of shape m
        neg_track: int of shape o
        neg_album: int of shape o
        neg_artist: int of shape o
    Returns:
        pos_affinity: affinity of context to the next track of shape m.
        neg_affinity: affinity of context to the neg tracks of shape o.
    """
    context_embed = self.get_embeddings(album_context, artist_context)
    next_embed = self.get_embeddings(next_album, next_artist)
    neg_embed = self.get_embeddings(neg_album, neg_artist)

    # The affinity of the context to the other track is simply the dot
    # product of each context embedding with the other track's embedding.
    # We also add a small boost if the album or artist match.
    pos_affinity = jnp.max(jnp.dot(next_embed, context_embed.T), axis=-1)
    pos_affinity = pos_affinity + 0.1 * jnp.isin(next_album, album_context)
    pos_affinity = pos_affinity + 0.1 * jnp.isin(next_artist, artist_context)

    neg_affinity = jnp.max(jnp.dot(neg_embed, context_embed.T), axis=-1)
    neg_affinity = neg_affinity + 0.1 * jnp.isin(neg_album, album_context)
    neg_affinity = neg_affinity + 0.1 * jnp.isin(neg_artist, artist_context)

    all_embeddings = jnp.concatenate(
        [context_embed, next_embed, neg_embed], axis=-2)
    all_embeddings_l2 = jnp.sqrt(
        jnp.sum(jnp.square(all_embeddings), axis=-1))

    context_self_affinity = jnp.dot(jnp.flip(
        context_embed, axis=-2), context_embed.T)
    next_self_affinity = jnp.dot(jnp.flip(
        next_embed, axis=-2), next_embed.T)
    neg_self_affinity = jnp.dot(jnp.flip(neg_embed, axis=-2), neg_embed.T)

```

```
return (pos_affinity, neg_affinity,  
        context_self_affinity, next_self_affinity, neg_self_affinity,  
        all_embeddings_l2)
```

Let's dig into this a bit since this is the core of the model code. The first part is pretty straightforward: we convert the indices into embeddings by looking up the album and artist embedding and concatenating them as a single vector per track. It is in this location that you would add in other dense features by concatenation, or convert sparse features to embeddings as we have done.

The next part computes the affinity of the context to the next tracks. Recall that the context is composed of the first five tracks, and the next track is the rest of the playlist to be computed. We have several choices here for representing the context and computing the affinity.

For the affinity of the context, we have chosen the simplest form of affinity, that of a dot product. The other consideration is how we treat the context, since it is composed of five tracks. One possible way is to average all the context embeddings and use the average as the representation for the context. Another way is to find the track with the maximal affinity as the closest track in the context to that of the next track.

Details on various options can be found in [“Affinity Weighted Embedding”](#) by Jason Weston et al. We have found that if a user has diverse interests, finding the max affinity doesn't update the context embeddings in the same direction as the next track, as using the mean embedding does. In the case of playlists, the mean context embedding vector should function just as well because playlists tend to be on a single theme.

Notice that we compute the affinity for the negative tracks as well. This is because we want the next tracks to have more affinity to the context than the negative tracks. In addition to the affinity of the context and next tracks to the context, we also compute the L2 norm of the vectors as a way to regularize the model so it does not overfit on the training data. We also reverse the embedding vectors and compute what we call *self-affinity*, or the affinity of the context, next, and negative embeddings to themselves, simply by reversing the list of vectors and taking the dot product.

This does not exhaustively compute all the affinities of the set with itself; this again is left as an exercise for you as it builds intuition and skill in using JAX.

The results are then returned as a tuple to the caller.

Framing the Loss Function

Now, let's look at *train_spotify.py*. We will skip the boilerplate code and just look at the evaluation and training steps:

```
def eval_step(state, y, all_tracks, all_albums, all_artists):
    result = state.apply_fn(
        state.params,
        y["track_context"], y["album_context"], y["artist_context"],
        y["next_track"], y["next_album"], y["next_artist"],
        all_tracks, all_albums, all_artists)
    all_affinity = result[1]
    top_k_scores, top_k_indices = jax.lax.top_k(all_affinity, 500)
    top_tracks = all_tracks[top_k_indices]
    top_artists = all_artists[top_k_indices]
    top_tracks_count = jnp.sum(jnp.isin(
        top_tracks, y["next_track"])).astype(jnp.float32)
    top_artists_count = jnp.sum(jnp.isin(
        top_artists, y["next_artist"])).astype(jnp.float32)

    top_tracks_recall = top_tracks_count / y["next_track"].shape[0]
    top_artists_recall = top_artists_count / y["next_artist"].shape[0]

    metrics = jnp.stack([top_tracks_recall, top_artists_recall])

    return metrics
```

The first piece of code is the evaluation step. To compute the affinities of the entire corpus, we pass in the album and artist indices for every possible track in the corpus to the model and then sort them using `jax.lax.top_k`. The first two lines are the scoring code for recommending the next tracks from the context during recommendations. LAX is a utility library that comes with JAX that contains functions outside of the NumPy API that are handy to work with vector processors like GPUs and

TPUs. In the Spotify Million Playlist Dataset Challenge, one of the metrics is the recall@k at the artist and track level. For the tracks, the `isin` function returns the correct metric of the intersection of the next tracks and the top 500 scoring tracks of the corpus divided by the size of the set of next tracks. This is because the tracks are unique in the corpus. However, JAX's `isin` doesn't support making the elements unique, so for the artist recall metric, we might count artists in the recall set more than once. For the sake of computational efficiency, we use the multiple counts instead so that the evaluation might be computed quickly on the GPU so as not to stall the training pipeline. On a final evaluation, we might want to move the dataset to a CPU for a more accurate metric.

We use Weights & Biases again to track all the metrics, as depicted in [Figure 13-1](#). You can see how they fare with each other over several experiments:

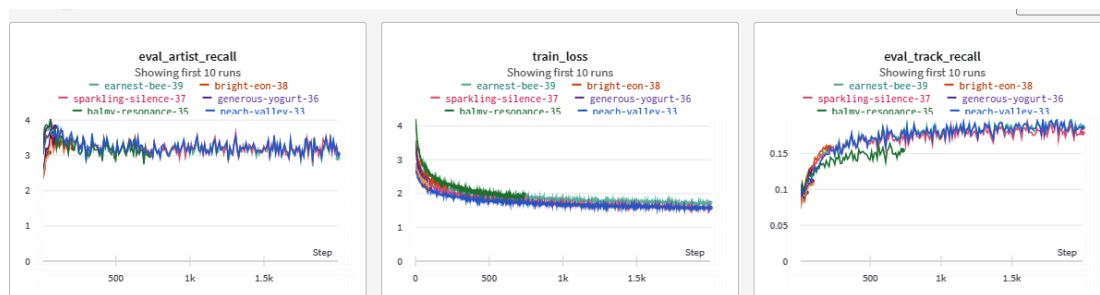


Figure 13-1. Weights & Biases experiment tracking

Next, we will look at the loss functions, another juicy part that you can experiment with in the exercises at the end of the chapter:

```
def train_step(state, x, regularization):
    def loss_fn(params):
        result = state.apply_fn(
            params,
            x["track_context"], x["album_context"], x["artist_context"],
            x["next_track"], x["next_album"], x["next_artist"],
            x["neg_track"], x["neg_album"], x["neg_artist"])
        pos_affinity = result[0]
        neg_affinity = result[1]
        context_self_affinity = result[2]
        next_self_affinity = result[3]
        neg_self_affinity = result[4]
        all_embeddings_l2 = result[5]
```

```

mean_neg_affinity = jnp.mean(neg_affinity)
mean_pos_affinity = jnp.mean(pos_affinity)
mean_triplet_loss = nn.relu(1.0 + mean_neg_affinity - mean_pos_affinity)

max_neg_affinity = jnp.max(neg_affinity)
min_pos_affinity = jnp.min(pos_affinity)
extremal_triplet_loss = nn.relu(
    1.0 + max_neg_affinity - min_pos_affinity
)

context_self_affinity_loss = jnp.mean(nn.relu(0.5 - context_self_affinity))
next_self_affinity_loss = jnp.mean(nn.relu(
    0.5 - next_self_affinity
))
neg_self_affinity_loss = jnp.mean(nn.relu(neg_self_affinity))

reg_loss = jnp.sum(nn.relu(all_embeddings_l2 - regularization))
loss = (extremal_triplet_loss + mean_triplet_loss + reg_loss +
        context_self_affinity_loss + next_self_affinity_loss +
        neg_self_affinity_loss)
return loss

grad_fn = jax.value_and_grad(loss_fn)
loss, grads = grad_fn(state.params)
new_state = state.apply_gradients(grads=grads)
return new_state, loss

```

We have several losses here, some directly related to the main task and others that help with regularization and generalization.

We initially started with the `mean_triplet_loss`, which is simply a loss that states that the positive affinity, or the affinity of the context tracks to the next tracks, should be one more than the negative affinity, or the affinity of the context tracks to the negative tracks. We will discuss how we experimented to obtain the other auxiliary loss functions.

Experiment tracking, depicted in [Figure 13-2](#), is important in the process of improving the model, as is reproducibility. We have tried as much as possible to make the training process deterministic by using random-number generators from JAX that are reproducible by using the same starting random-number generator seed.

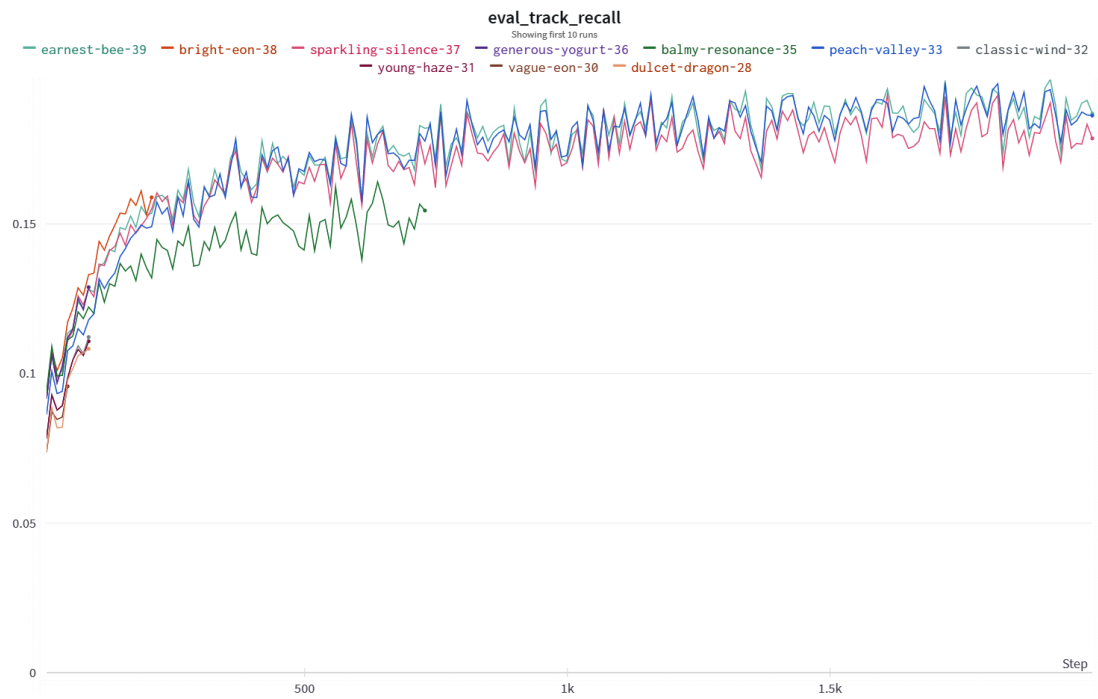


Figure 13-2. Track recall experiments

We started with the `mean_triplet_loss` and `reg_loss`, which is the regularization loss as a good baseline. These two losses simply make sure that the mean positive affinity of the context to the next track is one more than the negative affinity of the context to the negative tracks, and that the L2 norm of the embeddings does not exceed the regularization thresholds. These correspond to the metrics that did the worst. Notice that we do not run the experiment for the entire dataset. This is because for rapid iteration, it might be faster to just run on a smaller number of steps first and compare before interleaving occasionally with longer runs that use the entire dataset.

The next loss we added was the `max_neg_affinity` and the `min_pos_affinity`. This loss was inspired in part by [“Efficient Coordinate Descent or Ranking with Domination Loss”](#) by Mark A. Stevens and [“Learning to Rank Recommendations with the \$k\$ -Order Statistic Loss”](#) by Jason Weston et al. However, we do not use the entire negative set but merely a subsample. Why? Because the negative set is noisy. Just because a user hasn’t added a particular track to a playlist doesn’t mean that the track is not relevant to the playlist. Maybe the user hasn’t heard the track yet, so the noise is due to lack of exposure. We also do not do the sampling step as discussed in the k -order statistic loss paper because sampling is CPU friendly but not GPU friendly. So we combine ideas from both papers and take the largest negative affinity and make it

one less than the smallest positive affinity. The addition of this loss on the extremal tracks from both the next and negative sets gave us the next boost in performance in our experiments.

Finally, we added the self-affinity losses. These ensure that tracks from the context and next track sets have affinities of at least 0.5 and that the negative track affinities are at most 0. These are dot-product affinities and are more absolute as opposed to the relative positive and negative affinities that make the positive affinity one more than the negative affinities. In the long run, they didn't help much, but they did help the model converge faster in the beginning. We left them in because they still offer some improvement on the evaluation metrics on the last training step. This wraps up the explanatory part of this “Putting It All Together” chapter. Now comes the fun part, the exercises!

Exercises

We offer a lot of exercises because playing with the data and code is helpful in building out your intuition about different loss functions and ways of modeling the user. Also, thinking about how to write the code allows you to improve your proficiency with using JAX. So we have a list of helpful exercises to try out that are fun and will help you understand the material provided in this book.

To wrap up this chapter, here are some interesting exercises to experiment with. Doing them should give you lots of intuition about loss functions and the way JAX works, as well as a feel for the experimental process.

Here are some easy exercises to start with:

- Try out different optimizers (e.g., ADAM, RMSPROP).
- Try changing the feature sizes.
- Add in duration as a feature (take care on normalization!).
- What if you use cosine distance for inference and dot product for training?
- Add in a new metric, like NDCG.

- Play with distribution of positive versus negative affinities in the loss.
- Hinge loss with the lowest next track and the highest negative track.

Continue exploring with these more difficult exercises:

- Try using the track names as features and see if they help generalize.
- What happens if you use a two-layer network for affinity?
- What happens if you use an LSTM to compute affinity?
- Replace track embeddings with correlation.
- Compute all the self-affinities in a set.

Summary

What does it mean to replace an embedding with a feature? In our example of positive and negative affinity, we used the dot product to compute the affinity between two entities, such as two tracks, $\mathbf{x}$ and $\mathbf{y}$. Rather than having the features as latent, represented by embeddings, an alternative is to manually construct features that represent the affinity between the two entities, $\mathbf{x}$ and $\mathbf{y}$. As covered in [Chapter 9](#), this can be log counts or Dice correlation coefficient or mutual information.

Some kind of counting feature can be made and then stored in a database. Upon training and inference, the database is looked up for each entity $\mathbf{x}$ and $\mathbf{y}$, and the affinity scores are then used instead of or in conjunction with the dot product that is being learned. These features tend to be more precise but have less recall than an embedding representation. The embedding representation, being of low rank, has the ability to generalize better and improve recall. Having counting features is synergistic with embedding features because we can simultaneously improve precision with the use of precise counting features and, at the same time, improve recall with the help of low-rank features like embeddings.

For computing all n^2 affinities of tracks to other tracks in a set, consider using JAX's `vmap` function. `vmap` can be used to convert code that, for example, computes one track's affinity with all the other tracks and makes it run for all tracks versus all other tracks.

We hope that you have enjoyed playing with the data and code and that your skill in writing recommender systems in JAX has improved considerably after trying these exercises!